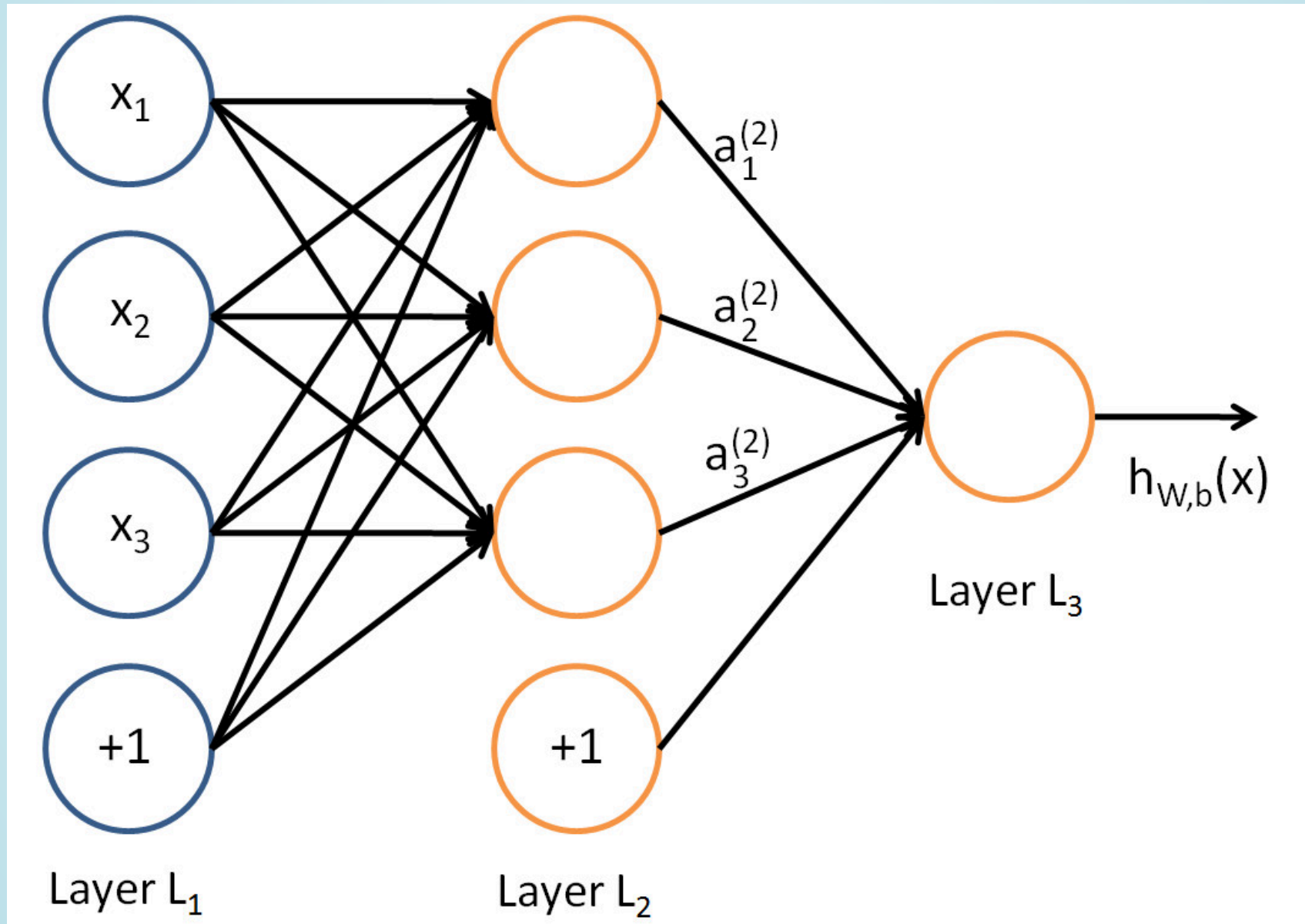


FROM NEURONS TO NEURAL NETWORKS

THE BASIC NEURAL NETWORK

- Following with the brain analogy one can combine (artificial) neurons to create better learners.
- A simple artificial neural network is created by two types of modifications to the basic Artificial Neuron.
 - *Stacking several neurons* insteads of just one.
 - Adding an additional layer of neurons, which is call a *hidden layer*,
- This yields a system where *the output of a neuron can be the input of another* in many different ways.

AN ARTIFICIAL NEURAL NETWORK



THE ARCHITECTURE OF ANN

In this figure, we have used circles to also denote the inputs to the network.

- Circles labeled +1 are *bias units*, and correspond to the intercept term.
- The leftmost layer of the network is called the *input layer*.
- The rightmost layer of the network is called the *output layer*.
- The middle layer of nodes is called the *hidden layer*, because its values are not observed in the training set.

Bias nodes are not counted when stating the neuron size.

With all this in mind our example neural network has three layers with:

- 3 input units (not counting the bias unit),
- 3 hidden units,
- 1 output unit.

HOW AN ANN WORKS

An ANN is a predictive model (a *learner*) whose properties and behaviour can be well characterized.

- It operates through a process known as *forward propagation*, which encompasses the information flow from the input layer to the output layer.
- Forward propagation is performed by composing a series of linear and non-linear (activation) functions.
- These are characterized (parametrized) by their *weights* and *biases*, that need to be *learnt*.
 - This is done by *training the ANN*.

TRAINING THE ANN

- The training process aims at finding *the best possible parameter values* for the learning task defined by the functions. This is done by
 - Selecting an appropriate (convex) loss function,
 - Finding weights that minimize the total cost function (avg. loss).
- This is usually done using some iterative optimization procedure such as *gradient descent*.
 - This requires evaluating derivatives in a huge number of points.
 - Such high number may be reduced by *Stochastic Gradient Descent*.
 - The evaluation of derivatives is simplified thanks to *Backpropagation*.

FORWARD PROPAGATION

The process that encompasses the computations required to go from the input values to the final output is known as *forward propagation*.

For the ANN with 3 input values and 3 neurons in the hidden layer we have:

1. Each node, $a_i^{(2)}$ of the hidden layer operates on all nodes of the input values

$$\begin{aligned}a_1^{(2)} &= f(\theta_{10}^{(1)} + \theta_{11}^{(1)} x_1 + \theta_{12}^{(1)} x_2 + \theta_{13}^{(1)} x_3) \\a_2^{(2)} &= f(\theta_{20}^{(1)} + \theta_{21}^{(1)} x_1 + \theta_{22}^{(1)} x_2 + \theta_{23}^{(1)} x_3) \\a_3^{(2)} &= f(\theta_{30}^{(1)} + \theta_{31}^{(1)} x_1 + \theta_{32}^{(1)} x_2 + \theta_{33}^{(1)} x_3)\end{aligned}$$

2. Output of hidden layer is transformed by the activation function:

$$h_{\Theta}(x) = a_1^{(3)} = f(\theta_{10}^{(2)} + \theta_{11}^{(2)} a_1^{(2)} + \theta_{12}^{(2)} a_2^{(2)} + \theta_{13}^{(2)} a_3^{(2)})$$

A COMPACT NOTATION (1)

Let $z_i^{(l)}$ denote the total weighted sum of inputs to unit i in layer l :

$$z_i^{(2)} = \theta_{i0}^{(1)} + \theta_{i1}^{(1)} x_1 + \theta_{i2}^{(1)} x_2 + \theta_{i3}^{(1)} x_3,$$

the output becomes: $a_i^{(l)} = f(z_i^{(l)})$.

Extending the activation function $f(\cdot)$ to apply elementwise to vectors:

$$f([z_1, z_2, z_3]) = [f(z_1), f(z_2), f(z_3)],$$

we can write the previous equations more compactly as:

$$\begin{aligned} z^{(2)} &= \Theta^{(1)} x \\ a^{(2)} &= f(z^{(2)}) \\ z^{(3)} &= \Theta^{(2)} a^{(2)} \\ h_{\Theta}(x) &= a^{(3)} = f(z^{(3)}) \end{aligned}$$

A COMPACT NOTATION (2)

More generally, recalling that we also use $a^{(1)} = x$ to also denote the values from the input layer,

Given layer l 's activations $a^{(l)}$, we can compute layer $l + 1$'s activations $a^{(l+1)}$ as:

$$z^{(l+1)} = \Theta^{(l)} a^{(l)}$$

$$a^{(l+1)} = f(z^{(l+1)})$$

A COMPACT NOTATION (3)

This can be used to provide a matrix representation for the weighted sum of inputs of all neurons:

$$z^{(l+1)} = \begin{bmatrix} z_1^{(l+1)} \\ z_2^{(l+1)} \\ \vdots \\ z_{s_{l+1}}^{(l)} \end{bmatrix} = \begin{bmatrix} \theta_{10}^{(l)} & \theta_{11}^{(l)} & \theta_{12}^{(l)} & \cdots & \theta_{1s_l}^{(l)} \\ \theta_{20}^{(l)} & \theta_{21}^{(l)} & \theta_{22}^{(l)} & \cdots & \theta_{2s_l}^{(l)} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ \theta_{s_{l+1}0}^{(l)} & \theta_{s_{l+1}1}^{(l)} & \theta_{s_{l+1}2}^{(l)} & \cdots & \theta_{s_{l+1}s_l}^{(l)} \end{bmatrix} \cdot \begin{bmatrix} 1 \\ a_1^{(l)} \\ a_2^{(l)} \\ \vdots \\ a_{s_l}^{(l)} \end{bmatrix}$$

A COMPACT NOTATION (4)

So that, the activation is then:

$$a^{(l+1)} = \begin{bmatrix} a_1^{(l+1)} \\ a_2^{(l+1)} \\ \vdots \\ a_{s_{l+1}}^{(l)} \end{bmatrix} = f(z^{(l+1)}) = \begin{bmatrix} f(z_1^{(l+1)}) \\ f(z_2^{(l+1)}) \\ \vdots \\ f(z_{s_{l+1}}^{(l)}) \end{bmatrix}$$

EFICIENT FORWARD PROPAGATION

- The way input data is transformed, through a series of weightings and transformations, until the ouput layer is called *forward propagation*.
- By organizing parameters in matrices, and using matrix-vector operations, fast linear algebra routines can be used to perform the required calculations in a fast efficient way.

MULTIPLE ARCHITECTURES FOR ANN

- We have so far focused on a single hidden layer neural network of the example.
- One can, however build neural networks with many distinct architectures (meaning patterns of connectivity between neurons), including ones with multiple hidden layers.
- See [here the Neural Network Zoo](#).

MULTIPLE ARCHITECTURES FOR ANN

- We have so far focused on a single hidden layer neural network of the example
- One can build neural networks with many distinct architectures (meaning patterns of connectivity between neurons), including ones with multiple hidden layers.

MULTIPLE LAYER DENSE NETWORKS

- Most common choice is a n_l -layered network:
 - layer 1 is the input layer,
 - layer n_l is the output layer,
 - and each layer l is densely connected to layer $l + 1$.
- In this setting, to compute the output of the network, we can compute all the activations in layer L_2 , then layer L_3 , and so on, up to layer L_{n_l} , using equations seen previously.

FEED FORWARD NNS

- The type of NN described is called feed-forward *neural network* (FFNN), since
 - All computations are done by Forward propagation
 - The connectivity graph does not have any directed loops or cycles.